RWANDA TVET BOARD

Ministry of Education | Republic of Rwanda

TECHNICAL IMPLEMENTATION PLAN

RTB Schools GIS Monitoring & Intelligence System

Version 2.0 | March 2026 | Senior Engineering Edition

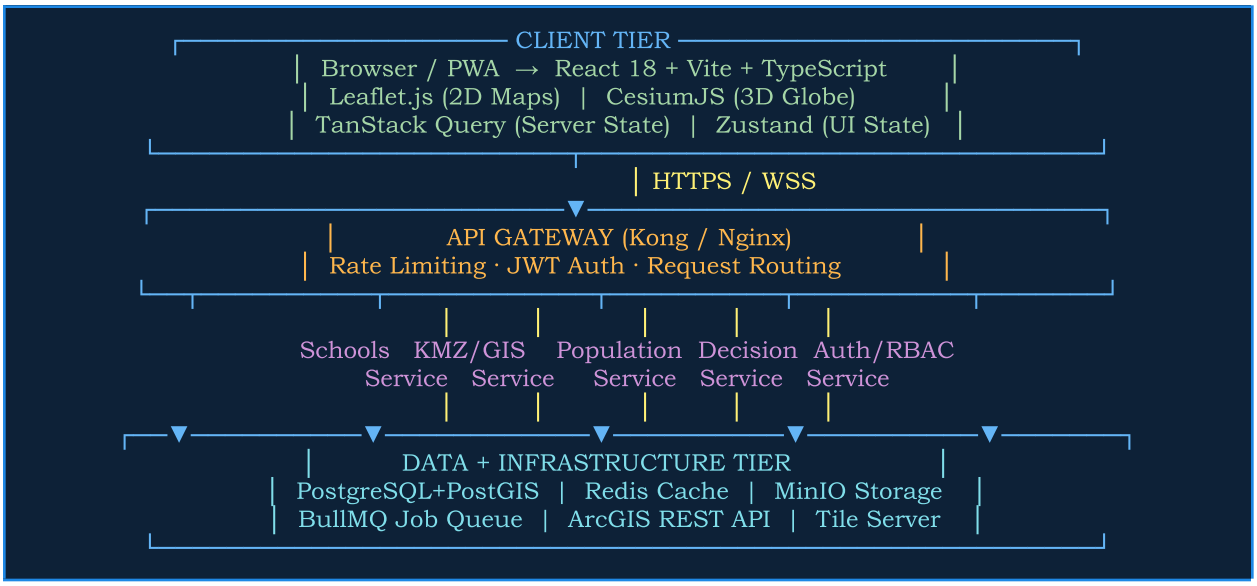
Document Type	Deep Technical Implementation Plan
System	Geospatial Intelligence & Infrastructure Monitoring Platform
Architecture	Microservices · PostGIS · CesiumJS · Drone Orthomosaic · Decision Engine
Tech Stack	React 18 + TypeScript · NestJS · PostgreSQL/PostGIS · GDAL · CesiumJS · Docker
Prepared For	Rwanda TVET Board — Engineering & Digital Transformation Division
Version	v1.0 — March 2026 | Classification: Internal Technical

01 System Architecture & Design Principles

1.1 Architecture Philosophy

The RTB GIS system is designed on a Domain-Driven Microservices architecture, isolating distinct bounded contexts — School Registry, Geospatial Processing, Population Analytics, and Decision Intelligence — into independently deployable services. This provides fault isolation, independent scalability, and clean ownership boundaries aligned with RTB's organizational units.

1.2 High-Level Architecture Diagram (Textual)



1.3 Design Principles

Principle	Application in RTB System
Spatial-First Design	Every school entity carries a POINT geometry from day one. All queries use spatial indexes. No school exists without a coordinate.
Offline-Resilient	Field inspectors can export school data + map tiles to a Progressive Web App (PWA). Sync happens on next connection.
Event-Driven Updates	KMZ upload triggers an async processing pipeline via BullMQ. No request blocking. Real-time progress via WebSocket.
Idempotent APIs	All POST/PUT endpoints are idempotent using client-generated UUIDs. Safe to retry on network failure.

Audit Everything	Every data mutation writes to an append-only audit_log table with actor, timestamp, old_value, and new_value JSON columns.
12-Factor App	All configuration via environment variables. Stateless services. Logs to stdout. Docker-native.

02 Complete Database Schema (PostGIS)

2.1 Schema Philosophy

The database is the single source of truth for all spatial and operational data. PostgreSQL 15 with PostGIS 3.4 extension. All geometry columns use EPSG:4326 (WGS84 — standard GPS coordinate system). Spatial indexes use GIST. Regular indexes use BTREE. JSON metadata in JSONB columns for schema flexibility.

2.2 Core Tables — Annotated DDL

Table 1: schools (Master Registry)

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS postgis_topology;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
```

```
CREATE TABLE schools (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  code              VARCHAR(20) UNIQUE NOT NULL,       -- RTB school code e.g. RTB-KGL-001
  name              VARCHAR(255) NOT NULL,
  school_type       VARCHAR(30) CHECK (school_type IN
('VTC','TSS','TVET_COLLEGE','POLYTECHNIC')),
  ownership         VARCHAR(20) CHECK (ownership IN ('PUBLIC','PRIVATE','FAITH_BASED','NGO')),
  status            VARCHAR(20) DEFAULT 'ACTIVE' CHECK (status IN
('ACTIVE','SUSPENDED','CLOSED')),
  rtb_license_no    VARCHAR(50),
  year_established  SMALLINT,
  last_inspection   DATE,
  -- LOCATION
  province          VARCHAR(50) NOT NULL,
  district          VARCHAR(50) NOT NULL,
  sector            VARCHAR(50) NOT NULL,
  cell              VARCHAR(50),
  village           VARCHAR(50),
  geom              GEOMETRY(POINT, 4326) NOT NULL,   -- GPS location
  -- METRICS
  total_area_ha     NUMERIC(8,2),                     -- land area in hectares
  built_area_m2     NUMERIC(10,2),                    -- total floor area
  -- CAPACITY
  student_capacity  INTEGER,
  current_enrollment INTEGER,
  male_students     INTEGER,
  female_students   INTEGER,
  trainer_count     INTEGER,
  -- METADATA
  metadata          JSONB DEFAULT '{}',
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW(),
  created_by        UUID REFERENCES users(id),
```

```
  is_deleted        BOOLEAN DEFAULT FALSE
);
CREATE INDEX idx_schools_geom     ON schools USING GIST(geom);
CREATE INDEX idx_schools_district ON schools(district);
CREATE INDEX idx_schools_sector   ON schools(sector);
CREATE INDEX idx_schools_status   ON schools(status);
```

Table 2: school_infrastructure

```
CREATE TABLE school_infrastructure (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  school_id       UUID NOT NULL REFERENCES schools(id) ON DELETE CASCADE,
  -- BUILDINGS COUNTS
  classrooms_total    INTEGER DEFAULT 0,
  classrooms_good     INTEGER DEFAULT 0,
  classrooms_fair     INTEGER DEFAULT 0,
  classrooms_poor     INTEGER DEFAULT 0,
  workshops_total     INTEGER DEFAULT 0,
  workshops_good      INTEGER DEFAULT 0,
  labs_total          INTEGER DEFAULT 0,
  dormitories_total   INTEGER DEFAULT 0,
  admin_blocks        INTEGER DEFAULT 0,
  latrines_male       INTEGER DEFAULT 0,
  latrines_female     INTEGER DEFAULT 0,
  -- SITE CONDITIONS
  terrain_type        VARCHAR(30) CHECK (terrain_type IN
('FLAT','GENTLE_SLOPE','STEEP','VALLEY','HILLTOP')),
  slope_percent       NUMERIC(5,2),                    -- % slope from DEM
  flood_risk          VARCHAR(10) CHECK (flood_risk IN ('NONE','LOW','MEDIUM','HIGH')),
  landslide_risk      VARCHAR(10) CHECK (landslide_risk IN ('NONE','LOW','MEDIUM','HIGH')),
  road_distance_km    NUMERIC(6,2),                    -- distance to nearest paved road
  road_type           VARCHAR(20),                     -- paved / murram / footpath
  -- UTILITIES
  has_electricity     BOOLEAN DEFAULT FALSE,
  electricity_source  VARCHAR(30),                     -- grid / solar / generator / none
  has_water           BOOLEAN DEFAULT FALSE,
  water_source        VARCHAR(30),                     -- piped / borehole / rainwater / none
  has_internet        BOOLEAN DEFAULT FALSE,
  internet_type       VARCHAR(20),                     -- fiber / 4G / none
  -- CONDITION SCORES (1-10)
  structural_score    NUMERIC(3,1),
  roof_condition      VARCHAR(10) CHECK (roof_condition IN ('GOOD','FAIR','POOR','CRITICAL')),
  drainage_condition  VARCHAR(10) CHECK (drainage_condition IN ('GOOD','FAIR','POOR')),
  -- RENOVATION HISTORY
  year_built          SMALLINT,
  last_renovation_year SMALLINT,
  renovation_notes    TEXT,
  assessed_by         UUID REFERENCES users(id),
  assessed_at         TIMESTAMPTZ,
  created_at          TIMESTAMPTZ DEFAULT NOW(),
  updated_at          TIMESTAMPTZ DEFAULT NOW()
);
```

Table 3: school_geospatial (KMZ-derived spatial data)

```
CREATE TABLE school_geospatial (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  school_id       UUID NOT NULL REFERENCES schools(id) ON DELETE CASCADE,
  version         INTEGER NOT NULL DEFAULT 1,
  is_current      BOOLEAN DEFAULT TRUE,
  -- GEOMETRIES (all extracted from KMZ)
  boundary        GEOMETRY(POLYGON, 4326),              -- school land boundary
  buildings       GEOMETRY(MULTIPOLYGON, 4326),         -- all building footprints
  access_roads    GEOMETRY(MULTILINESTRING, 4326),      -- internal + approach roads
  hazard_zones    GEOMETRY(MULTIPOLYGON, 4326),         -- marked risk zones
  point_markers   GEOMETRY(MULTIPOINT, 4326),           -- labeled POIs
  -- COMPUTED METRICS (auto-populated on insert via trigger)
  boundary_area_m2   NUMERIC(12,2),
  buildings_area_m2  NUMERIC(12,2),
  green_area_m2      NUMERIC(12,2),             -- boundary - buildings - roads
  perimeter_m        NUMERIC(10,2),
  -- SOURCE TRACKING
  kmz_file_id     UUID REFERENCES kmz_uploads(id),
  source_crs      VARCHAR(20) DEFAULT 'EPSG:4326',
  raw_kml_text    TEXT,
  raw_geojson     JSONB,
  uploaded_by     UUID REFERENCES users(id),
  uploaded_at     TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_geo_boundary    ON school_geospatial USING GIST(boundary);
CREATE INDEX idx_geo_buildings   ON school_geospatial USING GIST(buildings);
CREATE INDEX idx_geo_school      ON school_geospatial(school_id) WHERE is_current=TRUE;
```

Table 4: population_data (ArcGIS cache)

```
CREATE TABLE population_data (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  admin_level     VARCHAR(20) NOT NULL,                 -- 'sector' | 'district' | 'cell'
  admin_name      VARCHAR(100) NOT NULL,
  district        VARCHAR(50),
  province        VARCHAR(50),
  geom            GEOMETRY(MULTIPOLYGON, 4326),
  total_pop       INTEGER,
  male_pop        INTEGER,
  female_pop      INTEGER,
  under5_pop      INTEGER,
  school_age_pop  INTEGER,                              -- 5-18 estimated
  tvet_age_pop    INTEGER,                              -- 15-30 estimated
  pop_density_km2 NUMERIC(10,2),
  source_year     SMALLINT,
  fetched_at      TIMESTAMPTZ DEFAULT NOW(),
  raw_feature     JSONB                                 -- original ArcGIS feature
);
CREATE INDEX idx_pop_geom     ON population_data USING GIST(geom);
CREATE INDEX idx_pop_district ON population_data(district);
```

Table 5: decision_assessments (Scoring Engine output)

```
CREATE TABLE decision_assessments (
```

```
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  school_id       UUID NOT NULL REFERENCES schools(id),
  -- SCORES (weighted components, 0-100)
  score_infrastructure  NUMERIC(5,2),
  score_age             NUMERIC(5,2),
  score_population      NUMERIC(5,2),
  score_terrain         NUMERIC(5,2),
  score_compliance      NUMERIC(5,2),
  score_accessibility   NUMERIC(5,2),
  total_score           NUMERIC(5,2),
  -- CLASSIFICATION
  priority_level  SMALLINT CHECK (priority_level BETWEEN 1 AND 4),
  priority_label  VARCHAR(20),                    -- CRITICAL/HIGH/MEDIUM/LOW
  -- RECOMMENDATIONS (structured JSON)
  recommendations JSONB DEFAULT '[]',
  renew_reasons   JSONB DEFAULT '[]',
  flags           JSONB DEFAULT '[]',
  -- POPULATION CONTEXT
  pop_500m        INTEGER,
  pop_1km         INTEGER,
  pop_2km         INTEGER,
  tvet_demand_ratio NUMERIC(8,4),
  -- AUDIT
  scored_by_engine  VARCHAR(20) DEFAULT 'v1.0',
  scored_at         TIMESTAMPTZ DEFAULT NOW(),
  approved_by       UUID REFERENCES users(id),
  approved_at       TIMESTAMPTZ
);
```

Table 6: kmz_uploads (File management)

```
CREATE TABLE kmz_uploads (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  school_id       UUID NOT NULL REFERENCES schools(id),
  original_name   VARCHAR(255) NOT NULL,
  storage_key     VARCHAR(500) NOT NULL,              -- MinIO object key
  file_size_bytes BIGINT,
  mime_type       VARCHAR(100),
  processing_status VARCHAR(20) DEFAULT 'PENDING'
                  CHECK (processing_status IN ('PENDING','PROCESSING','SUCCESS','FAILED')),
  processing_log  TEXT,
  extracted_features JSONB,                           -- summary of what was found
  job_id          VARCHAR(100),                       -- BullMQ job ID
  uploaded_by     UUID NOT NULL REFERENCES users(id),
  uploaded_at     TIMESTAMPTZ DEFAULT NOW(),
  completed_at    TIMESTAMPTZ
);
```

Table 7: audit_log (Immutable trail)

```
CREATE TABLE audit_log (
  id          BIGSERIAL PRIMARY KEY,
  entity_type VARCHAR(50) NOT NULL,
  entity_id   UUID NOT NULL,
```

```
  action      VARCHAR(20) NOT NULL,         -- CREATE / UPDATE / DELETE / VIEW
  actor_id    UUID REFERENCES users(id),
  actor_email VARCHAR(255),
  ip_address  INET,
  old_values  JSONB,
  new_values  JSONB,
  changed_fields TEXT[],
  occurred_at TIMESTAMPTZ DEFAULT NOW()
);
-- Partition by month for performance
-- CREATE TABLE audit_log_2026_03 PARTITION OF audit_log
--   FOR VALUES FROM ('2026-03-01') TO ('2026-04-01');
```

Key Spatial Queries (PostGIS)

```
-- 1. All schools within 5km of a given school
SELECT s2.name, ST_Distance(s1.geom::geography, s2.geom::geography)/1000 AS dist_km
FROM schools s1, schools s2
WHERE s1.id = $1 AND s1.id != s2.id
AND ST_DWithin(s1.geom::geography, s2.geom::geography, 5000)
ORDER BY dist_km;
```

```
-- 2. Population within buffer zones of a school
SELECT
  SUM(CASE WHEN ST_Within(p.geom, ST_Buffer(s.geom::geography, 500)::geometry)
      THEN p.tvet_age_pop ELSE 0 END) AS pop_500m,
  SUM(CASE WHEN ST_Within(p.geom, ST_Buffer(s.geom::geography, 2000)::geometry)
      THEN p.tvet_age_pop ELSE 0 END) AS pop_2km
FROM population_data p, schools s WHERE s.id = $1;
```

```
-- 3. Schools with no KMZ boundary uploaded
SELECT s.id, s.name, s.district FROM schools s
LEFT JOIN school_geospatial g ON g.school_id = s.id AND g.is_current = TRUE
WHERE g.id IS NULL AND s.is_deleted = FALSE;
```

03 KMZ Processing Engine — Async Pipeline

3.1 Why Async (Not Synchronous)

KMZ files can contain large polygon sets, embedded imagery overlays, and multiple nested folders. Synchronous processing would block the HTTP request for 10–60 seconds on large files. Instead, the system uses BullMQ (Redis-backed job queue) to process KMZ asynchronously — the upload returns instantly with a job ID, and the frontend polls status via WebSocket.

3.2 Upload API Endpoint

POST /api/v1/schools/:schoolId/kmz

Parameter	Detail
Method	POST multipart/form-data
Auth	Bearer JWT — requires role: ADMIN or DISTRICT_INSPECTOR
File field	kmz_file — accepts .kmz only (MIME: application/vnd.google-earth.kmz)
Max size	50MB (configured in NestJS FileInterceptor + Multer)
Response	202 Accepted { uploadId, jobId, statusUrl } — non-blocking
WebSocket event	kmz:progress:{schoolId} — streams { stage, percent, message } to client

3.3 Full Pipeline — Step by Step

STEP 1 — File Reception & Validation

```
// NestJS Controller
@Post(':schoolId/kmz')
@UseInterceptors(FileInterceptor('kmz_file', { limits: { fileSize: 50_000_000 } }))
async uploadKmz(
  @Param('schoolId') schoolId: string,
  @UploadedFile() file: Express.Multer.File,
  @Request() req,
) {
  // 1a. Validate MIME type
  if (!['application/vnd.google-earth.kmz',
        'application/zip'].includes(file.mimetype)) {
    throw new BadRequestException('File must be a .kmz file');
  }
  // 1b. Store raw file in MinIO object storage
  const storageKey = `kmz/${schoolId}/${uuid()}.kmz`;
  await this.storageService.put(storageKey, file.buffer);
```

```
  // 1c. Create DB record
  const upload = await this.kmzRepo.create({ schoolId, storageKey, ... });
  // 1d. Enqueue async job
  const job = await this.kmzQueue.add('process-kmz', {
    uploadId: upload.id, schoolId, storageKey
  }, { attempts: 3, backoff: { type: 'exponential', delay: 5000 } });
  return { uploadId: upload.id, jobId: job.id, status: 'QUEUED' };
}
```

STEP 2 — BullMQ Worker: Unzip KMZ

```
@Processor('kmz-processing')
export class KmzProcessor {
  @Process('process-kmz')
  async handle(job: Job<KmzJobData>) {
    const { uploadId, schoolId, storageKey } = job.data;
    await job.updateProgress({ stage: 'UNZIP', percent: 10 });

    // 2a. Download from MinIO
    const buffer = await this.storageService.get(storageKey);

    // 2b. Unzip using JSZip
    const zip = await JSZip.loadAsync(buffer);

    // 2c. Find primary KML file (may be in subfolder)
    const kmlEntry = Object.values(zip.files).find(f =>
      f.name.endsWith('.kml') && !f.dir
    );
    if (!kmlEntry) throw new Error('No KML found inside KMZ');

    // 2d. Extract embedded images if any (for drone overlays)
    const imageEntries = Object.values(zip.files).filter(f =>
      /\.(jpg|jpeg|png|tiff)$/i.test(f.name)
    );
    const kmlText = await kmlEntry.async('text');
```

STEP 3 — KML XML Parsing

```
    await job.updateProgress({ stage: 'PARSE_KML', percent: 25 });

    // 3a. Parse XML to JS object tree
    const parsed = await parseStringPromise(kmlText, {
      explicitArray: false,
      mergeAttrs: true,
      normalize: true
    });

    // 3b. Flatten all Placemarks (handles nested Folders)
    const placemarks = this.extractAllPlacemarks(parsed.kml.Document);
```

STEP 4 — Geometry Classification & Extraction

```
    await job.updateProgress({ stage: 'EXTRACT_GEOMETRY', percent: 40 });
```

```
const geometries = { boundary: null, buildings: [], roads: [], points: [] };

for (const pm of placemarks) {
  const name = (pm.name || '').toLowerCase();
  const style = pm.styleUrl || pm.Style?.id || '';

  if (pm.Polygon) {
    const polygon = this.parsePolygon(pm.Polygon);
    // Classify by name convention or style
    if (name.includes('boundary') || name.includes('limit') || name.includes('fence')) {
      geometries.boundary = polygon;
    } else if (name.includes('building') || name.includes('block')) {
      geometries.buildings.push({ name: pm.name, geom: polygon });
    } else {
      geometries.buildings.push({ name: pm.name, geom: polygon }); // default
    }
  } else if (pm.LineString || pm.MultiLineString) {
    geometries.roads.push(this.parseLineString(pm.LineString || pm.MultiLineString));
  } else if (pm.Point) {
    geometries.points.push({ name: pm.name, geom: this.parsePoint(pm.Point) });
  }
}
```

STEP 5 — GDAL Conversion to GeoJSON (Server-side)

```
await job.updateProgress({ stage: 'CONVERT_GEOJSON', percent: 60 });

// Write KML to temp file, run ogr2ogr
const tmpKml  = `/tmp/${uuid()}.kml`;
const tmpJson = `/tmp/${uuid()}.geojson`;
fs.writeFileSync(tmpKml, kmlText);

await exec(`ogr2ogr -f GeoJSON ${tmpJson} ${tmpKml} -t_srs EPSG:4326`);

const geoJson = JSON.parse(fs.readFileSync(tmpJson, 'utf8'));
fs.unlinkSync(tmpKml); fs.unlinkSync(tmpJson); // cleanup
```

STEP 6 — PostGIS Storage with Auto-computed Metrics

```
await job.updateProgress({ stage: 'STORE_POSTGIS', percent: 80 });

// Insert into school_geospatial
// PostGIS triggers auto-compute area, perimeter
await this.db.query(`
  INSERT INTO school_geospatial (
    school_id, boundary, buildings, access_roads, point_markers,
    boundary_area_m2, buildings_area_m2, perimeter_m,
    kmz_file_id, raw_geojson, uploaded_by)
  VALUES ($1,
    ST_GeomFromGeoJSON($2),
    ST_GeomFromGeoJSON($3),
    ST_GeomFromGeoJSON($4),
```

```
      ST_GeomFromGeoJSON($5),
      ST_Area(ST_GeomFromGeoJSON($2)::geography),
      ST_Area(ST_GeomFromGeoJSON($3)::geography),
      ST_Perimeter(ST_GeomFromGeoJSON($2)::geography),
      $6, $7, $8)
    `, [schoolId, boundary, buildings, roads, points, uploadId, geoJson, userId]);

    // Mark previous geospatial as not current
    await this.db.query(
      'UPDATE school_geospatial SET is_current=FALSE WHERE school_id=$1 AND id!=$2',
      [schoolId, newGeoId]
    );
```

STEP 7 — Trigger Decision Engine Re-score

```
    await job.updateProgress({ stage: 'TRIGGER_SCORING', percent: 90 });
    await this.decisionQueue.add('rescore-school', { schoolId });

    // Notify frontend via WebSocket
    this.wsGateway.emit(`kmz:done:${schoolId}`, { uploadId, status: 'SUCCESS' });
    await job.updateProgress({ stage: 'COMPLETE', percent: 100 });
  }
}
```

04 Frontend Architecture — React + GIS

4.1 Project Structure (Feature-Sliced Design)

```
src/
├── app/                # App-level: router, providers, global styles
│   ├── router.tsx
│   └── providers.tsx     # TanStack Query, Auth, WebSocket providers
├── features/
│   ├── dashboard/        # National schools overview feature
│   │   ├── components/   # NationalMap, StatsPanel, SchoolTable
│   │   ├── hooks/        # useNationalSchools, useSchoolFilters
│   │   └── index.tsx
│   ├── school-detail/    # Individual school monitoring feature
│   │   ├── components/
│   │   │   ├── SchoolMap2D.tsx      # Leaflet component
│   │   │   ├── SchoolMap3D.tsx      # CesiumJS component
│   │   │   ├── KmzUploader.tsx      # Drag-drop + progress
│   │   │   ├── SchoolForm.tsx       # Registration form
│   │   │   ├── InfraPanel.tsx       # Infrastructure details
│   │   │   ├── PopulationPanel.tsx  # Population analytics
│   │   │   └── DecisionPanel.tsx    # Priority + recommendations
│   │   └── index.tsx
│   ├── decision-engine/  # Analytics, comparisons, reports
│   └── admin/            # User management, system config
├── shared/
│   ├── api/            # TanStack Query hooks + axios client
│   ├── components/       # Reusable UI: Badge, Modal, Table, etc.
│   ├── hooks/          # useWebSocket, useDebounce, useGeolocation
│   ├── stores/         # Zustand stores: mapStore, filterStore
│   └── utils/          # geoHelpers, numberFormatters, kmzUtils
└── types/              # Shared TypeScript interfaces
```

4.2 State Management Architecture

State Type	Library	What it Manages
Server State	TanStack Query v5	All API data: schools list, school detail, GeoJSON, population, decisions. Auto-caching, background refetching, optimistic updates.
UI / Map State	Zustand (lightweight)	Active school, selected map layer, 2D/3D mode toggle, filter panel state, sidebar open/closed.
Form State	React Hook Form + Zod	School registration forms, infrastructure forms. Zod schema validates on submit and on change.

Real-time State	Native WebSocket + Zustand	KMZ processing progress (stage, percent), notification badge count, live score updates.

4.3 National Dashboard Map (Leaflet)

```
// NationalMap.tsx — simplified
const NationalMap: React.FC = () => {
  const { data: schools } = useQuery({ queryKey: ['schools'], queryFn: fetchSchools });
  const { setActiveSchool } = useMapStore();

  const colorByPriority = (level: number) => ({
    1: '#B71C1C', // CRITICAL — red
    2: '#F57F17', // HIGH     — amber
    3: '#1565C0', // MEDIUM   — blue
    4: '#2E7D32', // LOW      — green
  }[level] ?? '#9E9E9E');

  return (
    <MapContainer center={[-1.95, 29.87]} zoom={9}>
      <TileLayer url='https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png' />
      <MarkerClusterGroup>
        {schools?.map(school => (
          <CircleMarker key={school.id}
            center={[school.lat, school.lng]}
            radius={8}
            fillColor={colorByPriority(school.priorityLevel)}
            onClick={() => setActiveSchool(school.id)}>
            <Popup><SchoolPopupCard school={school} /></Popup>
          </CircleMarker>
        ))}
      </MarkerClusterGroup>
    </MapContainer>
  );
};
```

4.4 Individual School 3D Map (CesiumJS)

```
// SchoolMap3D.tsx
const SchoolMap3D: React.FC<{ schoolId: string }> = ({ schoolId }) => {
  const viewerRef = useRef<Cesium.Viewer | null>(null);
  const { data: geoJson } = useQuery({
    queryKey: ['school-geojson', schoolId],
    queryFn: () => fetchSchoolGeoJson(schoolId)
  });

  useEffect(() => {
    if (!viewerRef.current) {
      viewerRef.current = new Cesium.Viewer('cesium-container', {
        terrainProvider: Cesium.createWorldTerrain({ requestWaterMask: true }),
        baseLayerPicker: false,
        geocoder: false,
```

```
      animation: false, timeline: false, // clean UI for RTB
      shadows: true,                     // sun-based shadow for classrooms
    });
  }
  const viewer = viewerRef.current;

  if (!geoJson) return;

  // Load school boundary + buildings as GeoJSON
  const dataSource = new Cesium.GeoJsonDataSource();
  dataSource.load(geoJson, {
    stroke: Cesium.Color.fromCssColorString('#1E88E5'),
    fill: Cesium.Color.fromCssColorString('#1E88E5').withAlpha(0.3),
    strokeWidth: 3,
    clampToGround: false,  // allow extrusion
  }).then(ds => {
    viewer.dataSources.add(ds);
    // Extrude building polygons
    ds.entities.values.forEach(entity => {
      if (entity.polygon) {
        entity.polygon.extrudedHeight = new Cesium.ConstantProperty(5.5); // ~2 floors
        entity.polygon.material = Cesium.Color.fromCssColorString('#90CAF9').withAlpha(0.7);
      }
    });
    viewer.flyTo(ds, { duration: 2.5 }); // smooth camera navigation
  });

  // Overlay drone orthomosaic tiles
  if (geoJson.droneLayerUrl) {
    viewer.imageryLayers.addImageryProvider(
      new Cesium.UrlTemplateImageryProvider({
        url: geoJson.droneLayerUrl + '/{z}/{x}/{y}.png',
        minimumLevel: 14, maximumLevel: 22
      })
    );
  }
 }, [geoJson]);

 return <div id='cesium-container' style={{ width: '100%', height: '600px' }} />;
};
```

4.5 KMZ Uploader Component

```
// KmzUploader.tsx — with real-time progress via WebSocket
const KmzUploader: React.FC<{ schoolId: string }> = ({ schoolId }) => {
 const [progress, setProgress] = useState<UploadProgress | null>(null);
 const { socket } = useWebSocket();

 // Listen for processing updates
 useEffect(() => {
  socket.on(`kmz:progress:${schoolId}`, (data: KmzProgress) => {
    setProgress(data);
  });
  return () => socket.off(`kmz:progress:${schoolId}`);
```

```
  }, [schoolId]);

  const onDrop = useCallback(async (files: File[]) => {
    const file = files[0];
    const formData = new FormData();
    formData.append('kmz_file', file);
    setProgress({ stage: 'UPLOADING', percent: 5, message: 'Sending file...' });
    await api.post(`/schools/${schoolId}/kmz`, formData);
    // Progress continues via WebSocket events
  }, [schoolId]);

  const STAGES =
['UNZIP','PARSE_KML','EXTRACT_GEOMETRY','CONVERT_GEOJSON','STORE_POSTGIS','COMPLETE
'];

  return (
    <div>
      <Dropzone onDrop={onDrop} accept={{ 'application/vnd.google-earth.kmz': ['.kmz'] }}>
        {({ getRootProps, getInputProps, isDragActive }) => (
          <div {...getRootProps()} className={`upload-zone ${isDragActive ? 'active' : ''}`}>
            <input {...getInputProps()} />
            <MapPinIcon /> Drop .kmz file here or click to browse
          </div>
        )}
      </Dropzone>
      {progress && (
        <ProcessingPipeline stages={STAGES} current={progress.stage} percent={progress.percent} />
      )}
    </div>
  );
};
```

05 Population Analytics Engine

5.1 ArcGIS API Integration Service

```
// population.service.ts
@Injectable()
export class PopulationService {
  private readonly ARCGIS_URL =
   'https://services5.arcgis.com/deNm5epdmeZgcm16/arcgis/rest/services'
   + '/population_count/FeatureServer/4/query';

  async fetchAndCache(): Promise<void> {
    // Fetch all features (paginated — ArcGIS default max 1000/page)
    let offset = 0; const pageSize = 1000;
    let hasMore = true;

    while (hasMore) {
      const url = new URL(this.ARCGIS_URL);
      url.searchParams.set('outFields', '*');
      url.searchParams.set('where', '1=1');
      url.searchParams.set('f', 'geojson');
      url.searchParams.set('resultOffset', String(offset));
      url.searchParams.set('resultRecordCount', String(pageSize));
      url.searchParams.set('geometryType', 'esriGeometryPolygon');

      const response = await fetch(url.toString());
      const geoJson: FeatureCollection = await response.json();

      for (const feature of geoJson.features) {
        await this.upsertPopulationRecord(feature);
      }

      hasMore = geoJson.features.length === pageSize;
      offset += pageSize;
    }
    this.logger.log(`Population data sync complete`);
  }

  private async upsertPopulationRecord(feature: Feature): Promise<void> {
    const props = feature.properties;
    await this.db.query(`
      INSERT INTO population_data
        (admin_name, district, province, geom, total_pop,
         school_age_pop, tvet_age_pop, pop_density_km2, raw_feature)
      VALUES ($1,$2,$3, ST_GeomFromGeoJSON($4), $5,$6,$7,$8,$9)
      ON CONFLICT (admin_name, source_year)
      DO UPDATE SET total_pop=$5, tvet_age_pop=$7, fetched_at=NOW()
    `, [props.NAME, props.DISTRICT, props.PROVINCE,
        JSON.stringify(feature.geometry),
        props.TOTAL_POP,
        Math.round(props.TOTAL_POP * 0.31),  // Rwanda: 31% school-age
        Math.round(props.TOTAL_POP * 0.22),  // 22% TVET-age (15-30)
```

```
      props.POP_DENSITY, JSON.stringify(feature)],
    );
  }
}
```

5.2 Per-school Population Metrics Query

```
// Computes all population metrics for a school in one query
SELECT
  s.id AS school_id,
  s.name,
  -- Population rings
  COALESCE(SUM(CASE WHEN ST_DWithin(s.geom::geography, p.geom::geography, 500)
    THEN p.tvet_age_pop END), 0) AS pop_500m,
  COALESCE(SUM(CASE WHEN ST_DWithin(s.geom::geography, p.geom::geography, 1000)
    THEN p.tvet_age_pop END), 0) AS pop_1km,
  COALESCE(SUM(CASE WHEN ST_DWithin(s.geom::geography, p.geom::geography, 2000)
    THEN p.tvet_age_pop END), 0) AS pop_2km,
  -- Demand ratio: TVET-age per seat
  ROUND(
    COALESCE(SUM(CASE WHEN ST_DWithin(s.geom::geography, p.geom::geography, 2000)
      THEN p.tvet_age_pop END), 0)::numeric
    / NULLIF(s.student_capacity, 0), 2
  ) AS demand_ratio
FROM schools s
LEFT JOIN population_data p ON p.admin_level = 'sector'
WHERE s.id = $1
GROUP BY s.id, s.name, s.student_capacity;
```

5.3 Population Scheduler (Auto-refresh)

```
// Cron job: refresh population data every Sunday at 2am
@Cron('0 2 * * 0')
async refreshPopulation() {
  this.logger.log('Starting weekly population data sync from ArcGIS...');
  await this.populationService.fetchAndCache();
  // Re-score all schools after population update
  const allSchoolIds = await this.schoolRepo.findAllIds();
  for (const id of allSchoolIds) {
    await this.decisionQueue.add('rescore-school', { schoolId: id }, { delay: 500 });
  }
}
```

06 Decision Engine — Scoring Algorithm

6.1 Weighted Scoring Model

The Decision Engine is a deterministic, rule-based scoring system implemented in TypeScript. It is fully auditable — every score component is stored in the database alongside the final result, enabling RTB leadership to understand and challenge any classification.

```
// decision-engine.service.ts
interface ScoreResult {
  infrastructure: number;  // 0-100
  age:            number;
  population:     number;
  terrain:        number;
  compliance:     number;
  accessibility:  number;
  total:          number;  // weighted sum
  priority:       1 | 2 | 3 | 4;
  label:          'CRITICAL' | 'HIGH' | 'MEDIUM' | 'LOW';
  recommendations: Recommendation[];
  flags: Flag[];
}

const WEIGHTS = {
  infrastructure: 0.25,
  age:            0.15,
  population:     0.20,
  terrain:        0.20,
  compliance:     0.10,
  accessibility:  0.10,
};
```

```
scoreInfrastructure(infra: SchoolInfrastructure): number {
  // Base: structural score (1-10) → normalize to 0-100
  let score = (infra.structural_score || 5) * 10;
  // Penalties
  if (infra.roof_condition === 'POOR')     score -= 20;
  if (infra.roof_condition === 'CRITICAL') score -= 40;
  if (infra.drainage_condition === 'POOR') score -= 10;
  const poorRooms = infra.classrooms_poor / Math.max(infra.classrooms_total, 1);
  score -= poorRooms * 30;  // up to -30 for all rooms poor
  return Math.max(0, Math.min(100, score));
}
```

```
scoreAge(infra: SchoolInfrastructure): number {
  const year = new Date().getFullYear();
  const lastRenovation = infra.last_renovation_year || infra.year_built || 2000;
  const age = year - lastRenovation;
  // Score decreases linearly: 0 yrs=100, 5 yrs=90, 15 yrs=60, 30+ yrs=0
  if (age <= 3)  return 100;
```

```
  if (age <= 10) return 100 - ((age - 3) * 5.7);
  if (age <= 25) return 60 - ((age - 10) * 4);
  return Math.max(0, 60 - (age * 2));
}
```

```
scorePopulation(school: School, pop: PopMetrics): number {
  const demandRatio = pop.pop_2km / Math.max(school.student_capacity, 1);
  // High demand + low capacity = low score (school is overwhelmed)
  if (demandRatio > 10) return 10;   // Severely underserved
  if (demandRatio > 5)  return 35;
  if (demandRatio > 2)  return 60;
  if (demandRatio > 1)  return 80;
  return 100;                        // Well-matched capacity
}
```

```
scoreTerrain(infra: SchoolInfrastructure): number {
  let score = 100;
  const risks = { NONE: 0, LOW: 15, MEDIUM: 35, HIGH: 60 };
  score -= risks[infra.flood_risk] || 0;
  score -= risks[infra.landslide_risk] || 0;
  if (infra.slope_percent > 15) score -= 20;
  if (infra.slope_percent > 30) score -= 20;
  if (infra.terrain_type === 'STEEP') score -= 15;
  return Math.max(0, score);
}
```

```
generateRecommendations(scores: ScoreResult, school: School,
                        infra: SchoolInfrastructure, pop: PopMetrics): Recommendation[] {
  const recs: Recommendation[] = [];

  if (scores.infrastructure < 40)
    recs.push({ type: 'URGENT_RENOVATION', message: 'Structural condition is critical.',
                action: 'Commission professional engineering assessment within 30 days.' });

  if (infra.classrooms_poor > 0)
    recs.push({ type: 'CLASSROOM_REPAIR', message: `${infra.classrooms_poor} classrooms in poor
condition.`,
                action: 'Schedule repairs in next quarter budget cycle.' });

  if (pop.demand_ratio > 5)
    recs.push({ type: 'CAPACITY_EXPANSION', message: `Demand ratio ${pop.demand_ratio}x above
capacity.`,
                action: 'Plan additional classroom blocks or satellite site.' });

  if (scores.terrain < 40)
    recs.push({ type: 'RELOCATION_STUDY', message: 'High terrain or natural hazard risk.',
                action: 'Commission site risk assessment and identify alternative land.' });

  if (!infra.has_water)
    recs.push({ type: 'UTILITY_WATER', message: 'No water access.',
                action: 'Install borehole or connect to piped water supply.' });

  return recs;
```

```
}
```

07 REST API Design — Complete Endpoint Reference

7.1 API Conventions

- Base URL: https://api.rtb-gis.rw/api/v1
- Authentication: Bearer JWT in Authorization header
- Content-Type: application/json (except file uploads: multipart/form-data)
- Pagination: cursor-based using { cursor, limit, direction } query params
- Error format: { statusCode, error, message, details? }
- All IDs: UUID v4

7.2 Endpoint Reference

Method	Endpoint	Auth Role	Description
SCHOOLS			
GET	/schools	ANY	List all schools with filters: province, district, sector, priority, status. Returns paginated GeoJSON FeatureCollection.
POST	/schools	ADMIN	Create new school. Returns created school with UUID.
GET	/schools/:id	ANY	Full school record with infrastructure, current geospatial, latest assessment.
PATCH	/schools/:id	ADMIN / INSPECTOR	Partial update. Records audit trail with changed_fields.
GET	/schools/:id/geojson	ANY	Current GeoJSON FeatureCollection for this school (boundary, buildings, roads).
GET	/schools/:id/nearby	ANY	Schools within radius (default 5km). Query param: radius=km.
GET	/schools/:id/population	ANY	Population analytics for 500m, 1km, 2km buffers. Demand ratio.
GET	/schools/:id/assessment	ANY	Latest decision assessment with all score components and recommendations.
POST	/schools/:id/assessment	ADMIN / ANALYST	Manually trigger re-scoring. Returns job ID.
KMZ FILES			

POST	/schools/:id/kmz	ADMIN / INSPECTOR	Upload KMZ file. Returns 202 { uploadId, jobId } immediately.
GET	/schools/:id/kmz	ANY	List all KMZ uploads for school with processing status.
GET	/kmz/:uploadId/status	ANY	Processing status: { stage, percent, message, completedAt }.
DELETE	/kmz/:uploadId	ADMIN	Delete a KMZ upload and associated geospatial data.
POPULATION			
GET	/population	ANY	GeoJSON of all population features. Query: district, sector.
POST	/population/refresh	ADMIN	Trigger manual ArcGIS data sync. Returns job ID.
GET	/population/gaps	ANALYST	Sectors with high TVET-age population and no school within 3km.
ANALYTICS			
GET	/analytics/overview	ANY	National KPIs: total schools, by priority, by province, compliance rate.
GET	/analytics/comparison	ANALYST	Compare 2-5 schools across all metrics. Returns normalized comparison matrix.
GET	/analytics/heatmap	ANY	GeoJSON heatmap data: school density, coverage gaps, population demand.
GET	/analytics/export	ADMIN / ANALYST	Export report: format=pdf|excel|csv. Async — returns job ID.

08 Security Architecture

8.1 Authentication & Authorization

JWT Token Structure

```
{
  "sub": "user-uuid",
  "email": "inspector@rtb.gov.rw",
  "role": "DISTRICT_INSPECTOR",
  "district": "Gasabo",         // scopes data access
  "permissions": ["schools:read", "schools:write", "kmz:upload"],
  "exp": 1800,                  // 30 min access token
  "refreshExp": 604800          // 7 day refresh token
}
```

Role Permission Matrix

Permission	RTB Admin	Inspector	Analyst	Viewer
View all schools	YES	YES	YES	YES
Edit school data	YES	DISTRICT	NO	NO
Upload KMZ	YES	YES	NO	NO
Delete records	YES	NO	NO	NO
Run Decision Engine	YES	NO	YES	NO
Export reports	YES	YES	YES	NO
Manage users	YES	NO	NO	NO
Trigger ArcGIS sync	YES	NO	NO	NO

8.2 Security Controls

- HTTPS-only with TLS 1.3. All HTTP redirected to HTTPS
- API rate limiting: 100 req/min per JWT, 20 req/min unauthenticated (Kong gateway)
- KMZ file scanning: ClamAV antivirus scan before processing begins
- SQL injection prevention: parameterized queries only, no string concatenation in SQL
- XSS prevention: DOMPurify on all user-supplied text rendered in React
- Drone tile storage: presigned MinIO URLs with 30-minute expiry
- CORS: whitelist RTB frontend domains only
- Password hashing: bcrypt with work factor 12
- Audit log: immutable — no UPDATE or DELETE allowed on audit_log table (PG row security policy)

- Rwanda Data Protection Law compliance: no PII exported without ADMIN approval

09 DevOps, Deployment & Infrastructure

9.1 Docker Compose (Development)

```yaml
# docker-compose.yml
services:
  postgres:
    image: postgis/postgis:15-3.4
    environment:
      POSTGRES_DB: rtb_gis
      POSTGRES_USER: rtb_admin
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes: [pgdata:/var/lib/postgresql/data]
    ports: ['5432:5432']

  redis:
    image: redis:7-alpine
    ports: ['6379:6379']

  minio:
    image: minio/minio
    command: server /data --console-address ':9001'
    environment:
      MINIO_ROOT_USER: ${MINIO_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_PASS}
    ports: ['9000:9000', '9001:9001']

  backend:
    build: ./backend
    depends_on: [postgres, redis, minio]
    environment:
      DATABASE_URL: postgresql://rtb_admin:${DB_PASSWORD}@postgres/rtb_gis
      REDIS_URL: redis://redis:6379
      MINIO_ENDPOINT: http://minio:9000
    ports: ['3001:3001']

  frontend:
    build: ./frontend
    ports: ['3000:80']
    depends_on: [backend]

  tile-server:
    image: maptiler/tileserver-gl
    volumes: ['./tiles:/data']
    ports: ['8080:80']
```

9.2 Production Architecture (Cloud)

Component	Service	Specification

Frontend CDN	AWS CloudFront + S3	Static React build served via CDN. Cache: 1yr for assets, no-cache for index.html
API Gateway	AWS ALB + Kong	SSL termination, rate limiting, JWT validation, routing to microservices
Backend Services	ECS Fargate (auto-scaling)	3 tasks min, scales to 10 on load. CPU: 512, RAM: 1GB per task
Database	RDS PostgreSQL 15	db.t3.medium, PostGIS enabled, automated backups, Multi-AZ
Job Queue	ElastiCache Redis + BullMQ	Persistent job queue for KMZ processing and population sync
File Storage	AWS S3	KMZ archives, drone tiles (GeoTIFF / XYZ tiles), PDF reports
Tile Server	TileServer-GL on ECS	Serves drone orthomosaic tiles as XYZ. Pre-rendered per school at zoom 14-22
WebSocket	NestJS Gateway + ALB sticky sessions	Real-time KMZ processing progress, score updates
Monitoring	CloudWatch + Grafana	Error rates, API latency P95, queue depth, DB query duration

9.3 CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/deploy.yml
on: push: branches: [main]
jobs:
  test:
    - npm run lint
    - npm run type-check
    - npm run test:unit
    - npm run test:e2e
  build:
    - docker build -t rtb-backend:$SHA .
    - docker push ecr.aws/.../rtb-backend:$SHA
  migrate:
    - npx prisma migrate deploy  # run pending DB migrations
  deploy:
    - aws ecs update-service --cluster rtb-prod --service backend \
        --task-definition rtb-backend:$VERSION
    - aws cloudfront create-invalidation --paths '/*'
```

10 Sprint-by-Sprint Implementation Plan

23-Week Delivery — Agile Sprints (2 weeks each)

Sprint	Phase	Deliverables	Acceptance Criteria	Duration
S1	Foundation	Dev environment (Docker Compose), PostgreSQL+PostGIS schema (all 7 tables + indexes), NestJS scaffold, JWT auth, basic user CRUD, Prisma ORM setup	DB migrations run cleanly; API returns 401 without JWT; seeded test users login successfully	Weeks 1–2
S2	School Registry	School CRUD API (POST/GET/PATCH), validation with class-validator, spatial point storage, Rwanda provinces/districts seed data, audit log trigger, Postman collection	Can create school with GPS; school appears on PostGIS; audit log captures all changes	Weeks 3–4
S3	Infrastructure Forms	Infrastructure table CRUD, school_infrastructure API, React form (React Hook Form + Zod), all field validations, auto-save on blur	Form submits full infrastructure record; validation prevents negative values; data persists after reload	Weeks 5–6
S4	KMZ Engine Pt 1	MinIO integration, file upload endpoint (50MB max), JSZip unzip, xml2js KML parser, BullMQ queue setup, Redis connection, kmz_uploads table	Upload KMZ → file stored in MinIO → job appears in BullMQ → DB record created with status PENDING	Weeks 7–8
S5	KMZ Engine Pt 2	GDAL ogr2ogr integration, geometry classification, PostGIS storage, computed metrics trigger, WebSocket progress events, version history, KMZ uploader React component	Full pipeline completes successfully on sample RTB KMZ file; boundary + buildings visible in DB; WebSocket emits stage updates	Weeks 9–10
S6	2D Mapping	React national dashboard with Leaflet, CircleMarker clustering, color-coded by priority, filter panel (province/district/sector/priority), school popup card, GeoJSON polygon overlay for individual school, buffer circle visualization	All 3 filter types work simultaneously; clicking marker navigates to detail; boundary polygon renders from PostGIS GeoJSON	Weeks 11–12

S7	**3D Visualization**	CesiumJS integration, KML/GeoJSON loading, building extrusion, world terrain provider, flyTo animation, drone orthomosaic tile overlay, 2D/3D toggle, slope analysis visualization, shadow mode	CesiumJS renders school in 3D with terrain; buildings extruded; drone tiles overlay at zoom 18+; camera flies smoothly on school load	Weeks 13–15
S8	**Population Analytics**	ArcGIS API integration, paginated fetch, population_data table, upsert logic, weekly cron job, buffer-zone SQL query, per-school population panel in React, demand ratio display, coverage gap API	Population sync fetches all Rwanda sectors from ArcGIS; school detail shows pop_500m/1km/2km; gaps API identifies underserved sectors	Weeks 16–17
S9	**Decision Engine**	All 6 scoring functions, weighted aggregation, priority classification, recommendation generation, decision_assessments table, auto-trigger on data change, DecisionPanel React component, priority badge on map markers	Scoring produces consistent results on known test schools; all 4 priority levels achievable; recommendations include actionable text	Weeks 18–19
S10	**Analytics & Reports**	National KPIs dashboard (Recharts), school comparison tool (select 2–5 schools), PDF export (puppeteer), Excel export (exceljs), heatmap layer, filter presets saved to localStorage	PDF report downloads with school map screenshot, infra table, and decision summary; Excel includes all school data	Weeks 20–21
S11	**QA & UAT**	End-to-end tests (Playwright), load testing (k6: 200 concurrent users), RTB UAT sessions (3 sessions × 2 days), bug fixes, accessibility audit (WCAG 2.1 AA), bilingual labels (EN/Kinyarwanda toggle)	Zero P0/P1 bugs open at UAT sign-off; API P95 latency < 800ms at 200 users; all RTB UAT test cases passed	Weeks 22–23

11 Innovation Features & Future Roadmap

11.1 Innovations Built Into v1.0

Innovation	Implementation Detail
Event-Sourced KMZ History	Every KMZ upload creates a new versioned record. RTB can compare boundaries over time — see if a school land boundary changed between 2023 and 2026 KMZ uploads. A geometry diff API computes the area gained or lost.
Auto-Classification of KMZ Geometries	The KML parser uses a smart classification heuristic — matching placemark names against a configurable vocabulary (boundary/fence/muro = boundary; class/salle/bloc = building; route/road/voie = access road). RTB can add Kinyarwanda terms to the vocabulary without code changes.
Demand Pressure Heatmap	A PostGIS-powered heatmap visualizes where TVET-age population density is high but school coverage is low — displayed directly on the national map. Decision-makers can literally see which part of Rwanda is underserved at a glance.
School Risk Score Badge	Every school displays a composite risk badge (color + number) on the national map marker. Computed automatically; updates in real-time when new infrastructure data or population data is submitted.
Offline PWA for Inspectors	Field inspectors can cache a school's data and map tiles to their phone before visiting. The assessment form works offline. When connectivity returns, changes sync automatically using a conflict-free merge strategy.
Drone Tile Progressive Loading	Drone orthomosaics are served as Cloud-Optimized GeoTIFF (COG) through a TileServer-GL instance. Tiles load progressively from low to high resolution as the user zooms in — no waiting for full image to download.
Bilingual Interface	All UI labels, form fields, and recommendation messages are available in English and Kinyarwanda. A language toggle in the header switches immediately without page reload (i18next library).

11.2 Phase 2 Roadmap (Post-Launch)

- AI-Assisted Condition Detection: Train a computer vision model (YOLOv8) on drone imagery to auto-detect roof condition, broken windows, and unmaintained grounds — feeding scores directly into the Decision Engine
- Mobile Inspection App (React Native): A native iOS/Android app for RTB inspectors with camera integration, GPS tagging, and offline forms synced to the same backend
- Digital Twin Mode: For high-priority schools, build a CesiumJS 3D model from drone photogrammetry data — enabling virtual walkthroughs before renovation planning

- Budget Simulation Tool: Input available capital budget; system allocates across schools by priority score and estimated renovation cost, maximizing impact per Rwandan franc
- Integration with EMIS: Connect with Rwanda's Education Management Information System for automatic enrollment and staffing data import — eliminating duplicate data entry
- Inter-agency Sharing: Expose a secure read-only API for MINEDUC, World Bank, and development partner portals to access aggregated (non-PII) school infrastructure statistics

12 Testing Strategy & Quality Assurance

Test Type	Tool	Coverage Target	What is Tested
Unit Tests	Jest + ts-jest	≥ 85% line coverage	Decision Engine scoring functions, KMZ parser utilities, PostGIS query builders
Integration Tests	Jest + Supertest + TestContainers	All API endpoints	Full HTTP request → DB response cycle; uses real PostGIS container via TestContainers
E2E Tests	Playwright	Critical user journeys	Upload KMZ → verify map updates; submit form → verify decision score changes; export PDF → verify file downloads
Spatial Tests	Custom PostGIS fixtures	All spatial queries	Buffer queries return correct populations; ST_DWithin finds correct nearby schools; geometry area calculations within 0.1% tolerance
Load Tests	k6	200 concurrent users	API P95 < 800ms; 500 KMZ jobs/hour without queue overflow; national map GeoJSON < 2s at full school count
Security Tests	OWASP ZAP + npm audit	Zero critical CVEs	SQL injection, XSS, CSRF, broken auth, file upload bypass, JWT tampering

12.1 Key Test Scenarios (RTB-specific)

1. Upload a real Google Earth KMZ file exported from a Rwandan school site → verify boundary polygon area matches expected hectares within 2%
2. Submit infrastructure form with structural_score = 2, flood_risk = HIGH, year_built = 1998 → verify Decision Engine outputs priority level CRITICAL with renovation recommendation
3. Query population for a school in Kigali Gasabo → verify pop_2km value matches ArcGIS data within 5% margin

4. Login as District Inspector for Gasabo → attempt to edit school in Musanze → verify 403 Forbidden response (district scoping)
5. Upload 45MB KMZ file → verify queue processes it asynchronously → verify WebSocket emits all 7 stages → verify boundary stored in PostGIS
6. Load national dashboard with 300+ schools → verify Leaflet cluster renders within 3 seconds on standard laptop

A Appendix: Technology Versions & Dependencies

Package / Tool	Version	Purpose & Notes
FRONTEND		
React	18.3+	UI framework. Concurrent rendering for map updates without freezing
TypeScript	5.4+	Type safety. Shared types between frontend and backend via monorepo
Vite	5.x	Build tool. Sub-second HMR. Replaces CRA/Webpack
TanStack Query	v5	Server state management. Caching, background refetch, optimistic updates
Zustand	4.x	Client UI state. Lightweight (< 1KB). Map state, filter state
Leaflet + React-Leaflet	1.9 / 4.x	2D tile maps. National overview. School boundary polygons
CesiumJS	1.116+	3D globe. Terrain. KML. Building extrusion. Drone imagery tiles
React Hook Form + Zod	7.x / 3.x	Forms + validation. Zod schema shared with backend DTOs
Recharts	2.x	Charts for analytics dashboard. Responsive. Composable
React-Dropzone	14.x	KMZ file upload drag-and-drop UI
i18next	23.x	Bilingual EN/Kinyarwanda internationalization
BACKEND		
Node.js	20 LTS	Runtime. LTS for production stability
NestJS	10.x	Backend framework. Modules, DI, decorators. TypeScript native
Prisma ORM	5.x	DB access layer + migrations. PostGIS geometry via raw SQL where needed
BullMQ	5.x	Redis-backed job queue for async KMZ processing
JSZip	3.x	KMZ unzip in Node.js
xml2js	0.6.x	KML XML → JavaScript object parsing
Socket.io	4.x	WebSocket for real-time processing progress
Puppeteer	22.x	PDF report generation (headless Chrome)
ExcelJS	4.x	Excel export (.xlsx) with school data and analytics
INFRASTRUCTURE		

PostgreSQL + PostGIS	15 / 3.4	Spatial database. GIST indexes. Buffer queries. Area computations
Redis	7.x	BullMQ job queue backend + API response caching
MinIO / AWS S3	Latest	KMZ file storage. Drone tile storage. Presigned URL delivery
GDAL / ogr2ogr	3.8+	KML → GeoJSON conversion. Drone GeoTIFF tiling (gdal2tiles)
TileServer-GL	3.x	Serves drone orthomosaic XYZ tiles via HTTP
Docker + Compose	24 / v2	Containerization. Dev and prod environments

RTB Schools GIS Monitoring & Intelligence System

Technical Implementation Plan v1.0 | March 2026 | Confidential — NGA Delivery Studio Internal